

Lesson 7: Introduction to Structured Query Language (SQL)

Data Definition Language (DDL)

- The SQL language commands that enable the creation, modification or removal of database objects, constraints, relationships and indexes.

Data Management Language (DML)

- The SQL language commands that enable the retrieval, insert, update, deletion of data from a database.

The SQL Dialect

- ANSI SQL is the standard for SQL and is also adopted by ISO.
 - Different DBMS's may have slight differences in their SQL dialect, but the core remains the same.
 - These differences mean that different dialects of SQL may not be completely interoperable.

Data Definition Language (DDL)

Creating a Database

- We always need to create a database first, followed by the tables and other objects.
- The complexity of authentication varies by DBMS
 - Can be done using a username and password for some circumstances
 - More likely to use SSO or Active Directory for enterprise databases.

Database Schema

- This is the logical group of database objects that are related.
- While a database can have a single schema, it can also have multiple schemas related to specific end user groups or applications.
 - MariaDB doesn't support this concept.
 - Microsoft SQL Server for example does and it supports which users have access to which objects.

Fully Qualified Names

- A fully qualified name contains the database, schema (if applicable) and object name.

```
<database>.<schema>.<table>  
SELECT *  
FROM hfh.dbo.supporter
```

- It's the alternative to a partially or non-qualified name

```
<schema>.<table>  
SELECT *  
FROM hfh.supporter  
OR  
<table>  
USE hfh  
SELECT *  
FROM supporter
```


Data Types

- We've already covered this, but will spend more time in lab diving into data types.
- Data types can be a bit variable between DBMS's, but there are many similarities.

Create Table Example

```
CREATE TABLE IF NOT EXISTS
  hfh.petition(petition_id INT UNSIGNED NOT NULL PRIMARY KEY AUTO_INCREMENT
              ,supporter_id INT UNSIGNED NOT NULL
              ,campaign_id INT UNSIGNED NOT NULL
              ,petition_signed_date DATE
              ,petition_url VARCHAR(2048) #Practical limit
              ,create_timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP
              ,update_timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
              ,CONSTRAINT `fk_campaign_petition` FOREIGN KEY (campaign_id) REFERENCES hfh.campaign(campaign_id)
              ON DELETE CASCADE
              ON UPDATE RESTRICT
              ,CONSTRAINT `fk_supporter_petition` FOREIGN KEY (supporter_id) REFERENCES hfh.supporter(supporter_id)
              ON DELETE CASCADE
              ON UPDATE RESTRICT
              #Prevent duplicative petition signatures
              ,CONSTRAINT `unq_supporter_campaign` UNIQUE(supporter_id, campaign_id));
```

Altering and Dropping Tables

- You can alter tables to modify existing columns (names, datatypes, etc) and constraints or add new columns or constraints.
 - We want to use "alter" to refer to changes in tables rather than "update" in order to avoid confusion between DDL and DML commands.
- Tables can be removed from database by dropping them.
 - Again we want to use "drop" rather than "delete" when we talk about getting rid of a table.
- These commands require a lot of thought, especially when tables already contain data.
 - The output of the drop or alter commands can result in data loss.

Alter Table Example

```
# Renaming a column  
ALTER TABLE hfh.petition  
    RENAME COLUMN petition_url TO petition_website_address;
```

Drop Table Example

```
# Remove a table from the database.  
# I hope that it doesn't have data in it!  
DROP TABLE hfh.petition;
```

--

Creating Indexes

- Recall that indexes enable a database to be more performant in data retrieval.
- However, they are also used to enforce entity integrity (primary keys), referential integrity (foreign keys) and data integrity (ex: unique constraints)

```
CREATE INDEX `descriptive_index_name` ON table(column);
```

Dropping Indexes

- You can also drop an index if you need to.

```
DROP INDEX `descriptive_index_name`;
```

Data Management Language (DML)

Adding Data

- Without data there was almost no point in creating a database, so adding or "inserting" data is a critical operation.
 - This can happen directly in a SQL client (like HeidiSQL) or can be done via an API
- We should always be explicit the columns in which we're inserting data.

INSERT Syntax

```
#Being explicit about the columns the values refer to
INSERT INTO database.table(column1, column2, column2)
VALUES(#First Tuple of Values
      (1, 'Test', 'Example')
      #Second Tuple of Values
      ,(2, 'Another Test', 'Another Example')
      #Third Tuple of Values
      ,(3, 'Last Test', 'Last Example')
      );
```

Considering NULL and Optional Values

- You will need to replace missing values with NULL.
 - Recall that NULL is a value available to all data types and doesn't get quoted.
- If you're inserting completely NULL values for optional attributes, then consider leaving those columns out of your INSERT command.

Querying or Selecting Data

- It turns out that "SELECT" actually relates to the PROJECT operation in relational algebra.
 - It returns the specified columns in a query or all of the columns.
- The FROM clause specifies the table (sometimes view) from which you'd like to retrieve the data.

SELECT Syntax

#Using an asterisk (wildcard) specifies that you want to select all of the columns

```
SELECT *  
FROM database.table;
```

#Specifically naming the columns specifies you only want the results to include those you defined

```
SELECT column1  
       ,column2  
       ,column3  
FROM database.table;
```

Calculating Columns with SELECT

- You can calculate new columns that don't exist in a table using the SELECT command.
- These can be calculated using arithmetic like $+$, $-$, $*$, $/$, $^$ or other functions (to discuss next lesson).
- New columns should always be named with an alias

```
SELECT column1 * 2 as column1_2x  
FROM database.table;
```

Combining SELECT and INSERT

- This uses the concept of subquery (aka nested or inner queries).

```
INSERT INTO database.table_copy(column1
                                ,column2
                                ,column3)
    SELECT column1
           ,column2
           ,column3
    FROM database.table;
```

Updating Data

- While we want to add new data, it's almost guaranteed that we want to update data too.
- Like inserting data, the updates can be made directly in a SQL client, but also done through an API.

Deleting Data

- Sometimes we will want or need to delete one or more rows of data.

```
DELETE FROM database.table  
#If you don't include a WHERE you will delete all of the data!  
WHERE column1 = 1;
```


Transaction Control

- Enables you to break commands into transactions to be committed or rolled back (undone).
 - Depending on the DBMS you can rollback the previous transaction or to a specific point in time.
- Transactions are typically the the safest way to INSERT, UPDATE and DELETE data.

Transaction Control Example

```
#Being explicit and not using autocommit  
#Other languages use "BEGIN TRANSACTION"  
START TRANSACTION;
```

```
UPDATE database.table  
    SET column1 = 4  
        column2 = 'Updated all'  
        column3 = 'of the values'  
    WHERE column1 = 1;
```

```
COMMIT;
```

The WHERE Clause

- Used to add conditions to the SELECT command to limit the results set.
- Coincidentally, it's like the SELECT or RESTRICT operations from relational algebra.

```
SELECT *  
FROM database.table  
WHERE column1 >= 3;
```

WHERE Clause Comparison Operators

- All can be used for numeric values (decimal, int) and dates.
- Equal and and not equal can be used with text values.

Symbol	Meaning
=	Equal to
<	Less than
>	Greater than
<=	Less than or equal to
>=	Greater than or equal to
!= or <>	Not equal to

WHERE Clause with AND and OR

```
SELECT *  
FROM database.table  
WHERE column1 = 1  
      OR column2 = 'Last Test';
```

```
SELECT *  
FROM database.table  
WHERE column1 = 3  
      AND column2 = 'Last Test';
```

WHERE Clause and NOT

- Negates the results of a conditional expression.

```
SELECT *  
FROM database.table  
WHERE NOT(column1 = 3);
```

WHERE Clause Special Operators

Operator	Meaning
BETWEEN	Checks if values are between a range
IS NULL	Checks if the value is empty or null
LIKE	Checks for simple fuzzy matching of text
IN	Checks if values are part of a specified list
EXISTS	Checks whether a subquery returns any rows

Formatting Best Practices

- Use leading commas
- If you create a new column use an alias.
- Space is better, that is don't condense everything to a single line.
- Use uppercase letters for COMMAND words and CLAUSES.
- Use fully qualified names when practical.

Bad Example

```
SELECT column1,column2 ,column3 FROM database.table WHERE column1 > condition1
```

Good Example

```
SELECT column1
       ,column2
       ,column3
       ,column1 * 2 as column1_2x
FROM database.table
WHERE column1 > condition1
       AND column2 < condition2
```

Another Good Example

```
SELECT
    column1
    ,column2
    ,column3
    ,column1 * 2 as column1_2x
FROM
    database.table
WHERE
    column1 > condition1
    AND column2 < condition2
```